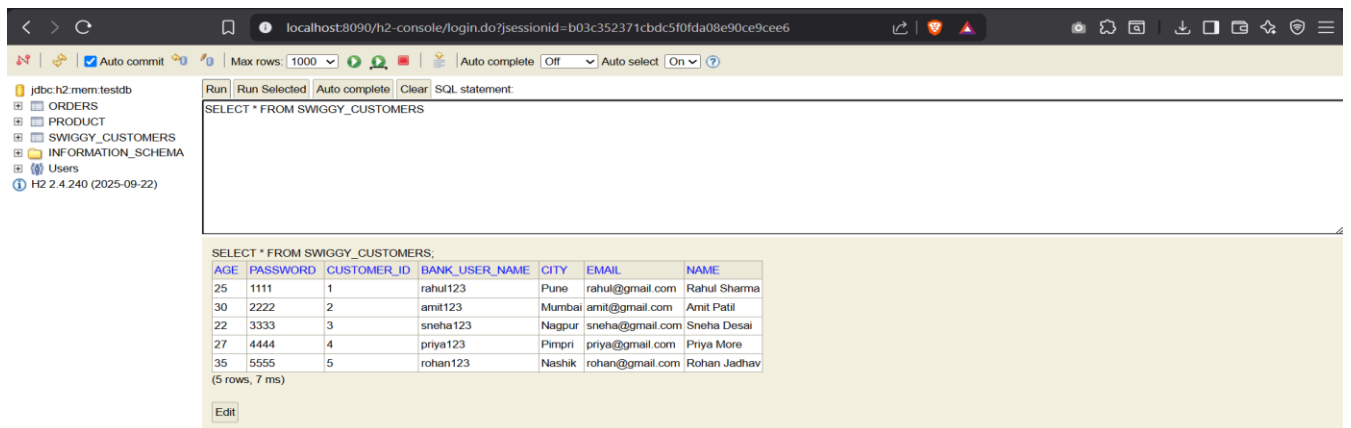
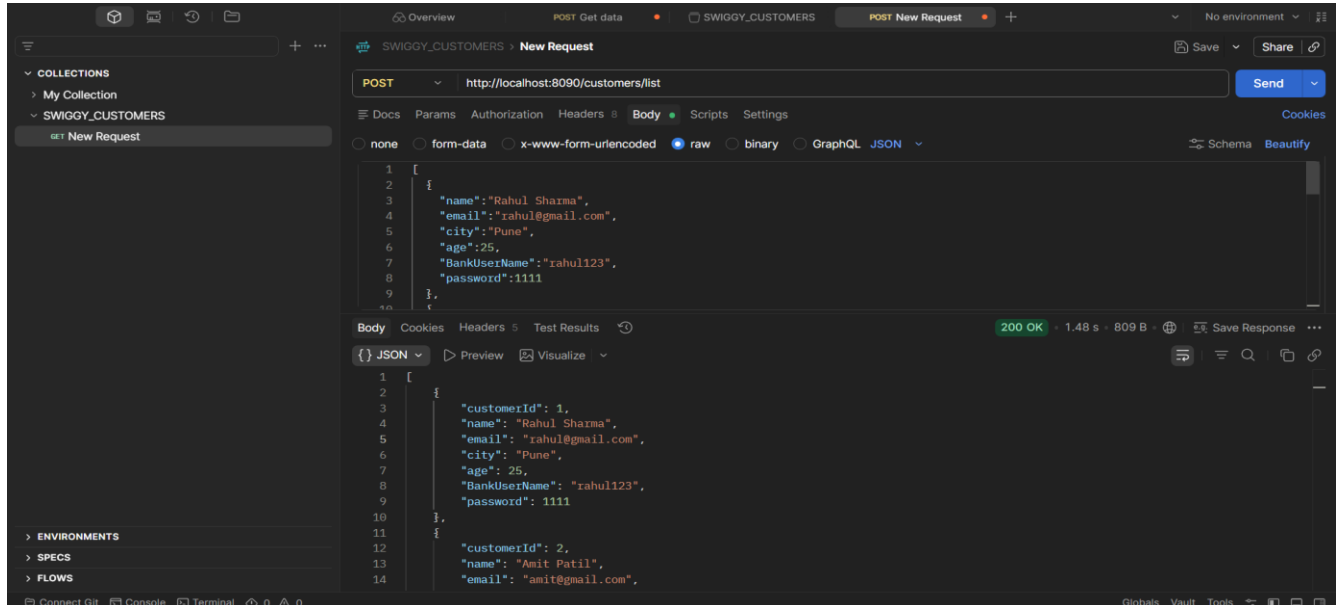


Name:- Atharv Prashant Satao
Class:- LY-IT-CORE

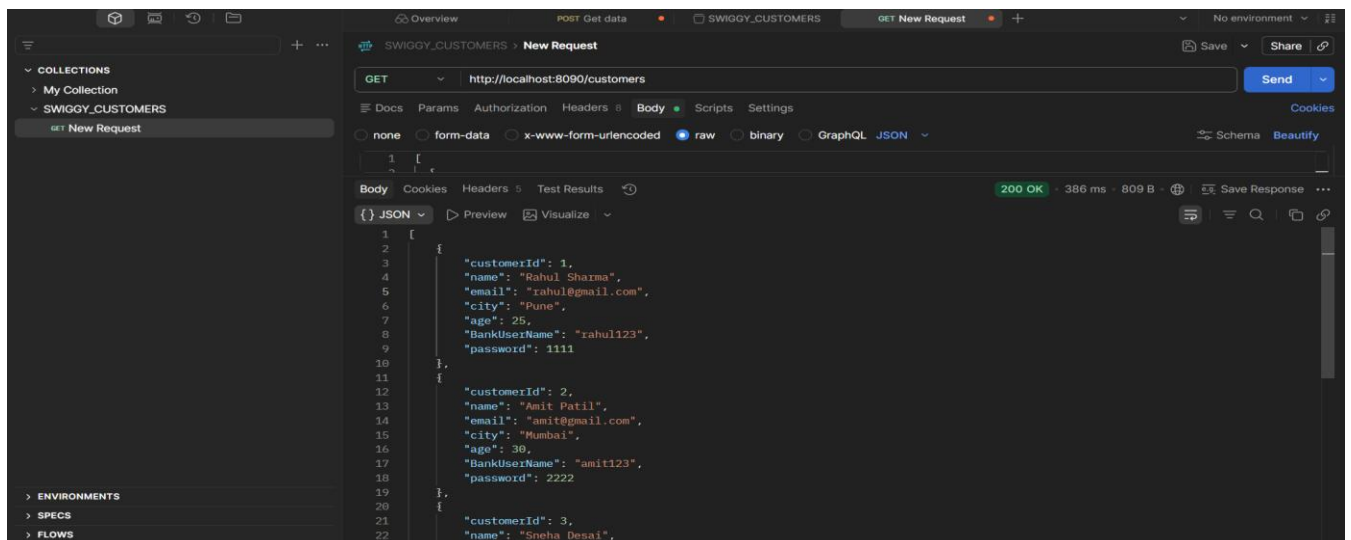
Enrollment No:- ADT23SOCB1512

For Customers

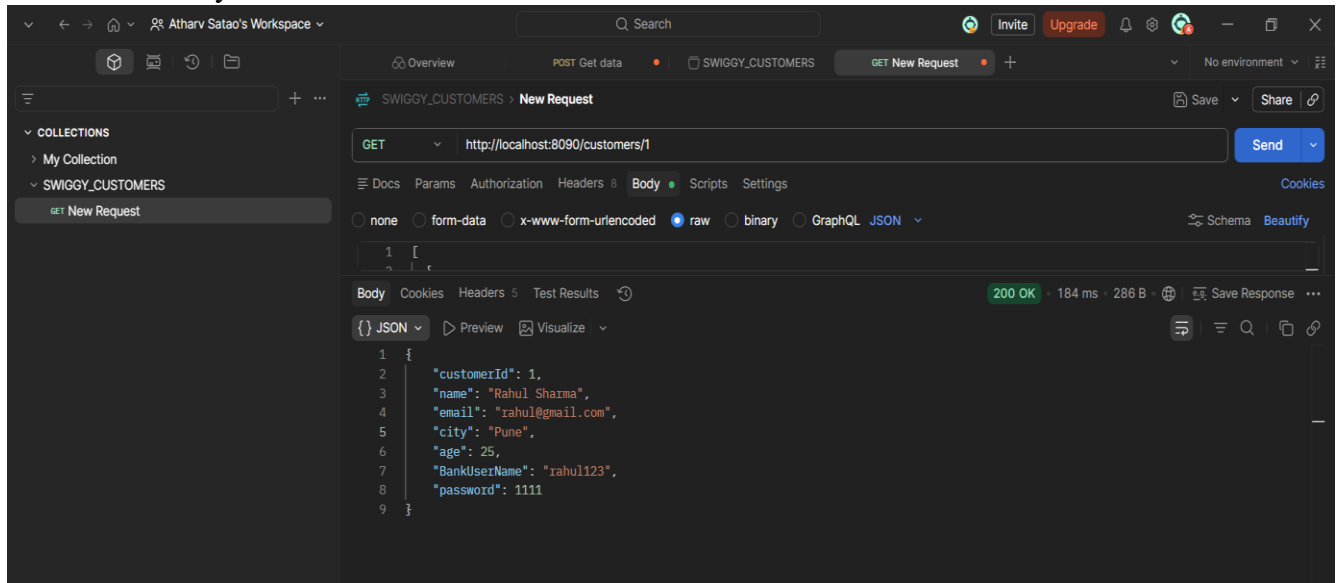
Insert List of Customers



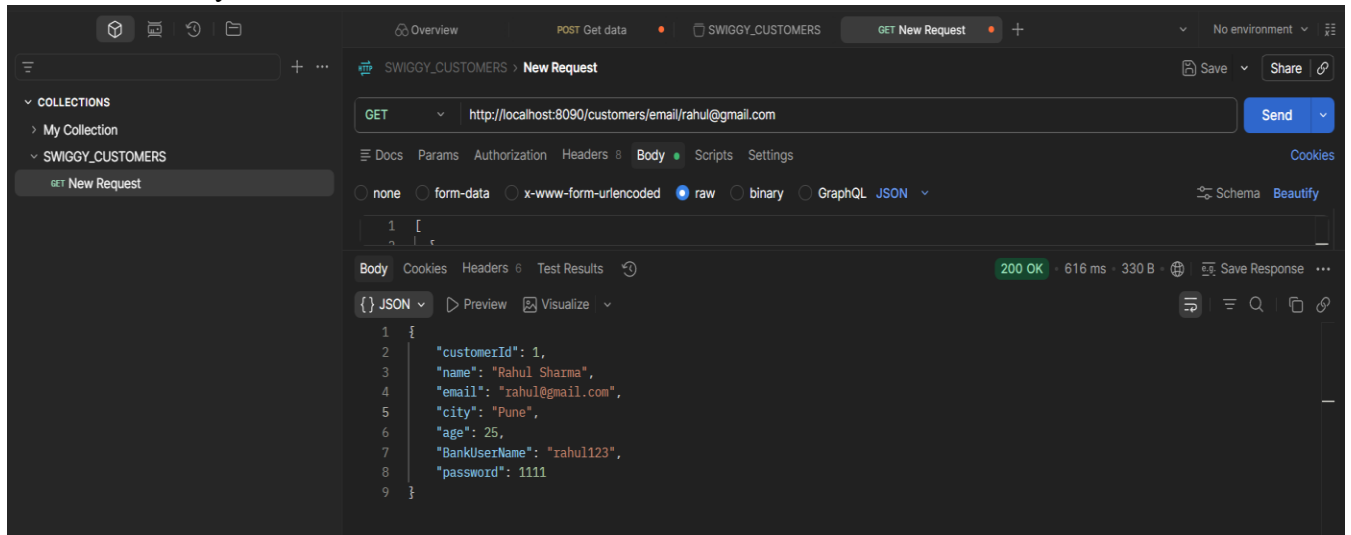
Get All Customers



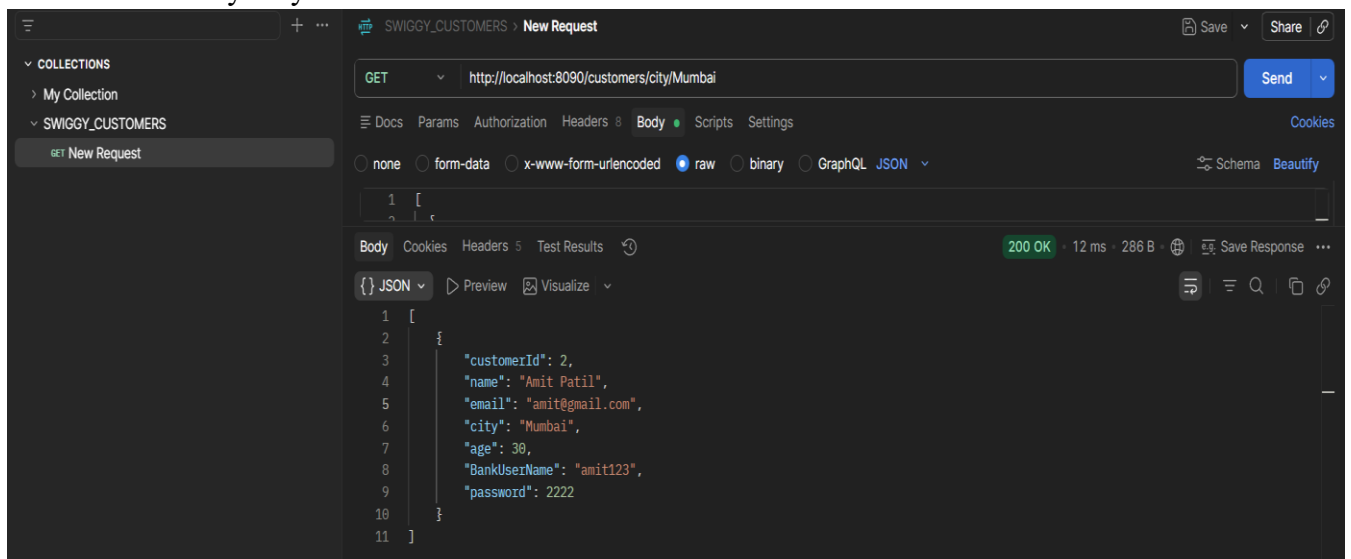
Get Customer By Id



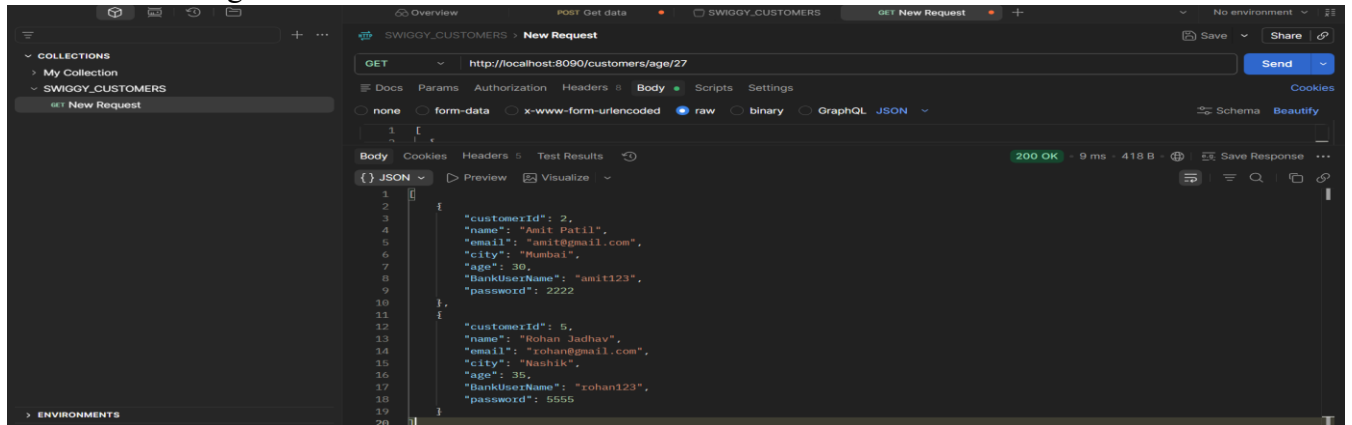
Get Customer By Email



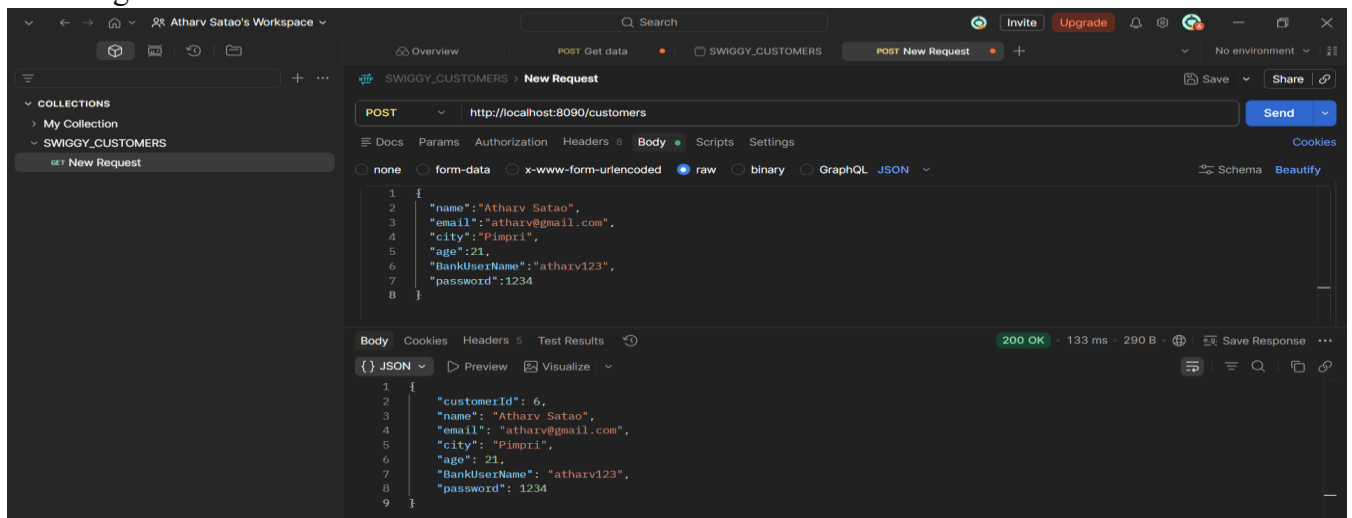
Get Customers By City



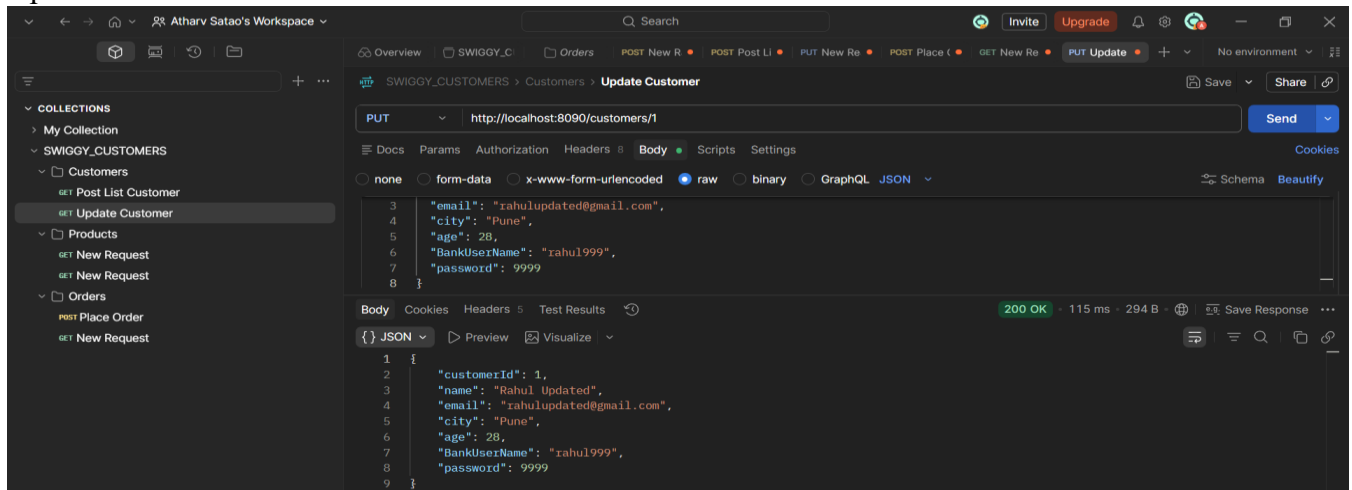
Get Customers Age Greater Than



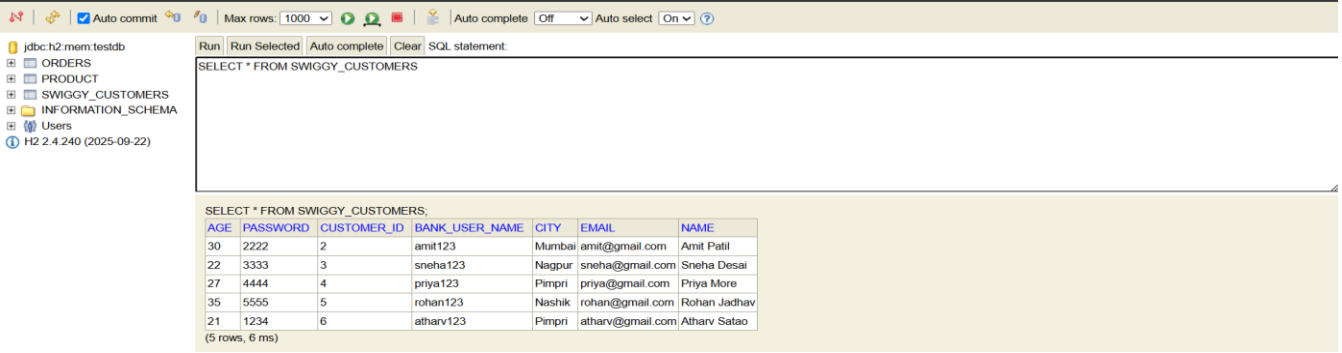
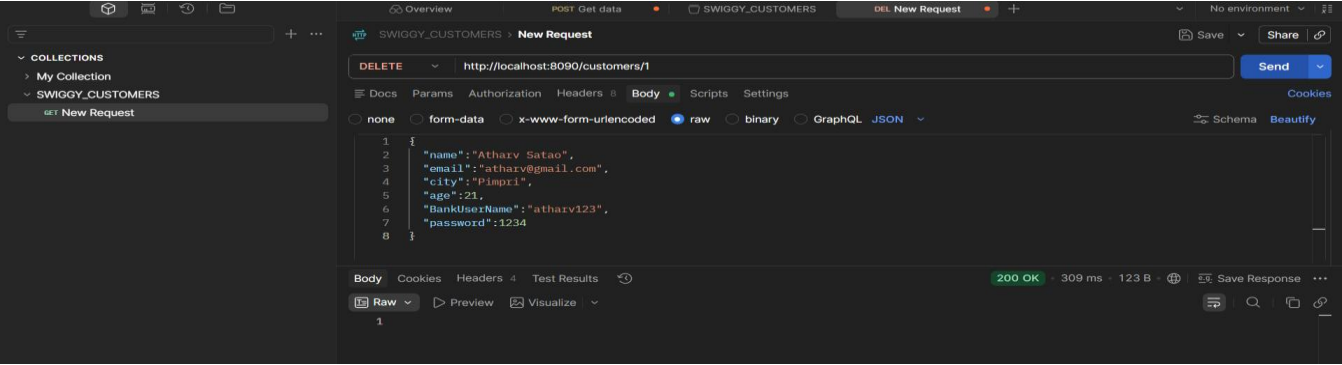
Add Single Customer



Update Customer



Delete Customer



For Products

Insert Products

Postman interface showing a POST request to `http://localhost:8090/products/list`. The request body is a JSON array with one product object:

```
[{"category": "Beverage", "productId": 3, "productName": "Cold Drink", "price": 60.0, "quantity": 100}]
```

The response is a 200 OK status with a response time of 187 ms and a body size of 518 B. The response body is a JSON array with three product objects:

```
[{"category": "FastFood", "productId": 1, "productName": "Pizza", "price": 299.0, "quantity": 20}, {"category": "FastFood", "productId": 2, "productName": "Burger", "price": 149.0, "quantity": 50}, {"category": "Dessert", "productId": 4, "productName": "Ice Cream", "price": 90.0, "quantity": 40}]
```

JDBC interface showing a SQL query: `SELECT * FROM PRODUCT;`

The results are displayed in a table with 5 columns: PRICE, QUANTITY, PRODUCT_ID, CATEGORY, and PRODUCT_NAME.

PRICE	QUANTITY	PRODUCT_ID	CATEGORY	PRODUCT_NAME
299.0	20	1	FastFood	Pizza
149.0	50	2	FastFood	Burger
60.0	100	3	Beverage	Cold Drink
90.0	40	4	Dessert	Ice Cream

(4 rows, 1 ms)

Get Product By Id

Postman interface showing a GET request to `http://localhost:8090/products/1`. The response is a 200 OK status with a response time of 53 ms and a body size of 243 B. The response body is a JSON object representing a single product:

```
{"category": "FastFood", "productId": 1, "productName": "Pizza", "price": 299.0, "quantity": 20}
```

Get Product By Category

Postman interface showing a GET request to `http://localhost:8090/products/category/FastFood`. The response is a 200 OK status with a response time of 279 ms and a body size of 335 B. The response body is a JSON array with two product objects:

```
[{"category": "FastFood", "productId": 1, "productName": "Pizza", "price": 299.0, "quantity": 20}, {"category": "FastFood", "productId": 2, "productName": "Burger", "price": 149.0, "quantity": 50}]
```

Product Deleted Successfully

The screenshot shows the Postman interface for a DELETE request. The request is configured with the method 'DELETE' and the URL 'http://localhost:8090/products/1'. The response is a 200 OK status, indicating the product was deleted successfully. The response body is '1 Product deleted Successfully'.

Request Details:

- Method: DELETE
- URL: http://localhost:8090/products/1
- Response: 200 OK, 106 ms, 192 B
- Body: 1 Product deleted Successfully

Update Product

The screenshot shows the Postman interface for a PUT request. The request is configured with the method 'PUT' and the URL 'http://localhost:8090/products/1'. The request body is a JSON object representing the product details. The response is a 200 OK status, indicating the product was updated successfully. The response body is a JSON object representing the updated product details.

Request Details:

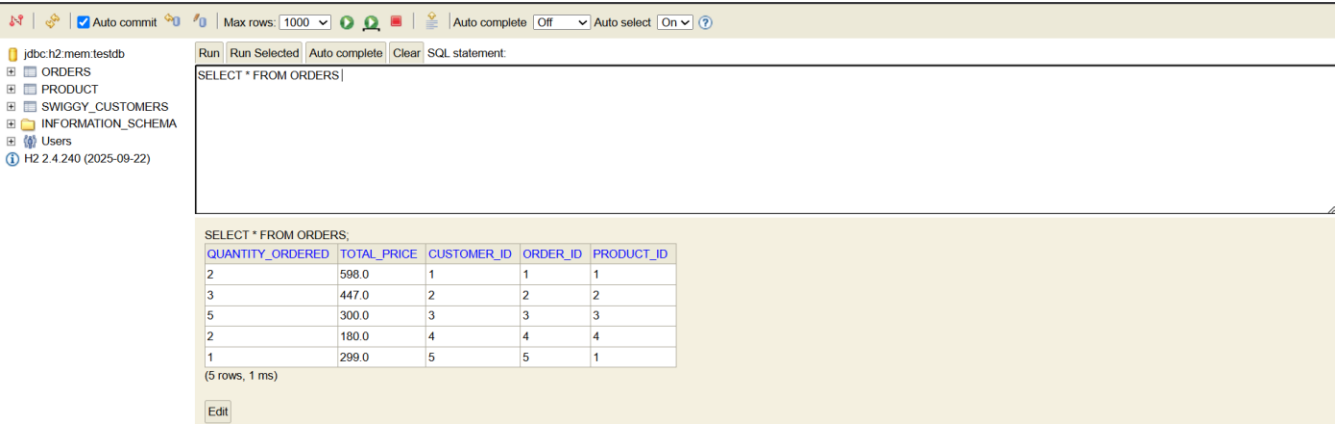
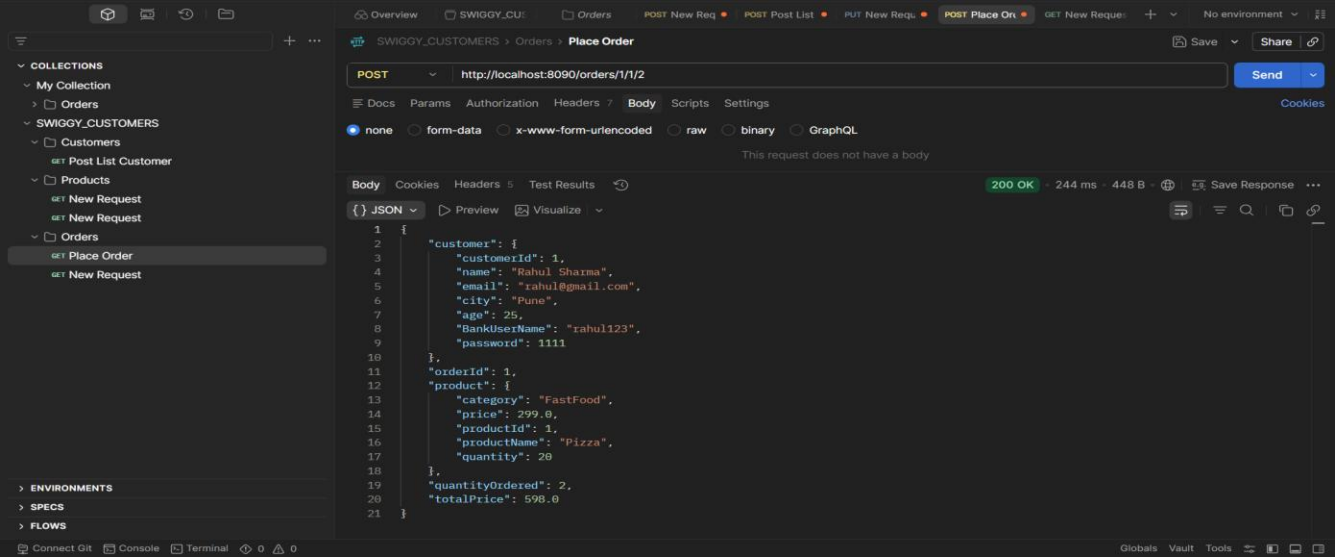
- Method: PUT
- URL: http://localhost:8090/products/1
- Body:

```
{  "productName": "Pizzas",  "price": 349,  "category": "FastFood",  "quantity": 15}
```
- Response: 200 OK, 18 ms, 244 B
- Body:

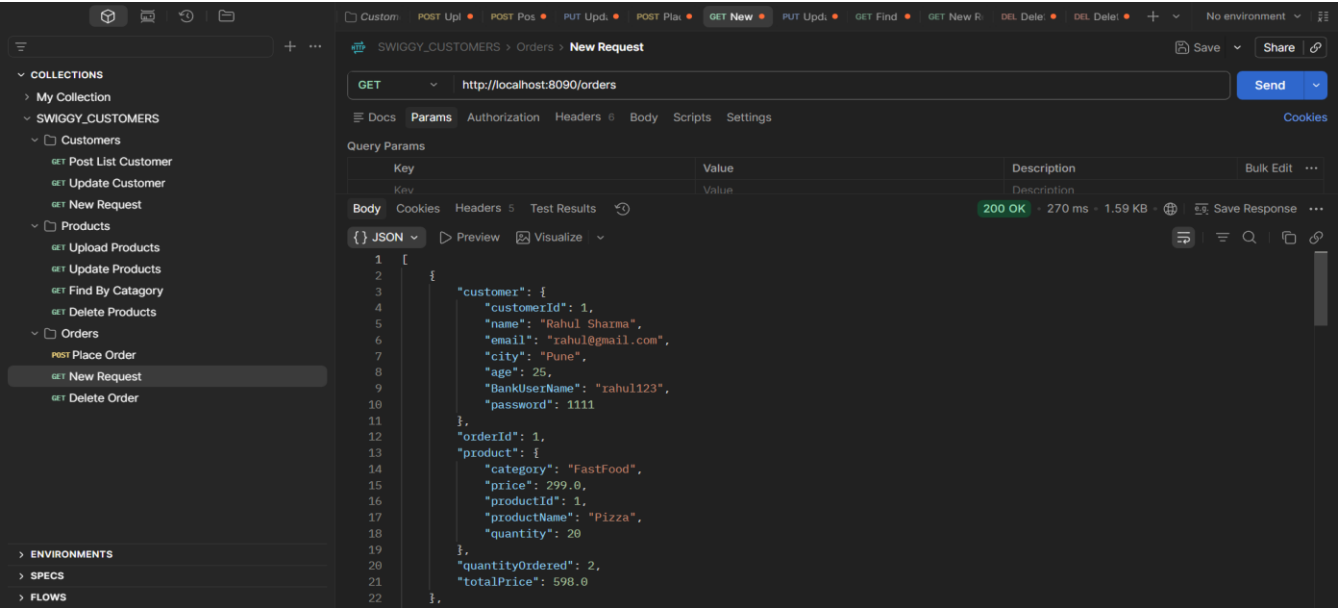
```
{  "category": "FastFood",  "price": 349.0,  "productId": 1,  "productName": "Pizzas",  "quantity": 15}
```

For Order

Place Order



Get All Orders



Update Order

The screenshot shows a Postman interface with a PUT request to `http://localhost:8090/orders/2/10`. The request body is a JSON object representing an order update. The response is a 200 OK status with a 61 ms response time and 449 B of data.

```
1 {
2   "customer": {
3     "customerId": 2,
4     "name": "Amit Patil",
5     "email": "amit@gmail.com",
6     "city": "Mumbai",
7     "age": 30,
8     "BankUserName": "amit123",
9     "password": 2222
10  },
11  "orderId": 2,
12  "product": {
13    "category": "FastFood",
14    "price": 149.0,
15    "productId": 2,
16    "productName": "Burger",
17    "quantity": 50
18  },
19  "quantityOrdered": 10,
20  "totalPrice": 1490.0
21 }
```

Delete Order

The screenshot shows a DELETE request to `http://localhost:8090/orders/4`. The response is a 200 OK status with a 40 ms response time and 190 B of data. The body of the response is "Order Deleted Successfully".

```
1 Order Deleted Successfully
```

Count Orders of Customer

The screenshot shows a GET request to `http://localhost:8090/orders/count/1`. The response is a 200 OK status with a 7 ms response time and 156 B of data. The body of the response is a JSON object with a single value of 1.

```
1 1
```


Total Amount Spend by Customer

The screenshot shows the Swagger UI for the 'SWIGGY_CUSTOMERS' API. The endpoint 'Total Amt of Cust' is selected under the 'Orders' collection. The method is 'GET' and the URL is 'http://localhost:8090/orders/amount/1'. The response body is shown as JSON:

```
{ "1": 598.0 }
```

. The status is '200 OK' with a response time of 47 ms and a body size of 160 B.

Total Revenue Generated

The screenshot shows the Swagger UI for the 'SWIGGY_CUSTOMERS' API. The endpoint 'Total Revenue Generated' is selected under the 'Orders' collection. The method is 'GET' and the URL is 'http://localhost:8090/orders/revenue'. The response body is shown as JSON:

```
{ "1": 1644.0 }
```

. The status is '200 OK' with a response time of 116 ms and a body size of 161 B.

The screenshot shows a SQL console window with a query and its results. The query is:

```
SELECT
  o.ORDER_ID,
  c.NAME,
  p.PRODUCT_NAME,
  p.CATEGORY,
  o.QUANTITY_ORDERED,
  o.TOTAL_PRICE
FROM ORDERS o
JOIN SWIGGY_CUSTOMERS c
  ON o.CUSTOMER_ID = c.CUSTOMER_ID
JOIN PRODUCT p
  ON o.PRODUCT_ID = p.PRODUCT_ID;
```

The results are shown in a table with 6 columns: ORDER_ID, NAME, PRODUCT_NAME, CATEGORY, QUANTITY_ORDERED, and TOTAL_PRICE. There are 5 rows of data.

ORDER_ID	NAME	PRODUCT_NAME	CATEGORY	QUANTITY_ORDERED	TOTAL_PRICE
1	Rahul Sharma	Pizza	FastFood	2	598.0
2	Amit Patil	Burger	FastFood	3	447.0
3	Sneha Desai	Cold Drink	Beverage	5	300.0
5	Rohan Jadhav	Pizza	FastFood	1	299.0

(4 rows, 13 ms)